MODULE 40 · DOMAIN 5 - OPERATIONAL EXCELLENCE AND AUTOMATION

# Infrastructure as Code

Choose declarative or imperative authoring, keep secrets out of templates, protect what must survive, and deploy the same stack to many accounts and Regions.

DOMAIN 5 of 5 EXAM WEIGHT 20% MODULE 40 of 52

admin@sthithapragna.com

# Declarative or imperative?

DECLARATIVE

## Describe the end state

- CloudFormation templates in YAML or JSON
- Engine works out the changes
- Change sets preview each update
- Easy to review, harder to abstract

IMPERATIVE

## Write code that builds it

- AWS CDK in TypeScript, Python, Java and more
- Loops, types, tests, reusable constructs
- Synthesises to CloudFormation
- SDK scripts alone keep no state

THE TIEBREAKER

The CDK is imperative to write but declarative to deploy, because it produces CloudFormation.

# CDK construct levels

| LEVEL | WHAT IT IS | EXAMPLE |
|---|---|---|
| L1 | One-to-one with a CloudFormation resource | CfnBucket |
| L2 | Sensible defaults and helper methods | Bucket with grantRead() |
| L3 patterns | Several resources wired together | An ALB-fronted Fargate service |
| Your constructs | Company standards packaged once | A compliant bucket with logging |

ON THE EXAM

Company-wide standards shipped as a construct library, reused by every team.

# Safe CloudFormation operations

- **Change sets** preview creates, modifications and replacements.
- **Drift detection** finds changes made outside IaC.
- **Drift-aware change sets** revert drift, previewed, since November 2025.
- **IaC generator and resource import** adopt existing resources.
- **Rollback** returns the stack to its last good state on failure.

> ON THE EXAM
>
> A change set that shows Replacement for a database is a warning, not a detail.

# Protecting resources that must survive

| CONTROL | PREVENTS |
| --- | --- |
| `DeletionPolicy: Retain or Snapshot` | Data lost when a resource leaves the stack |
| `UpdateReplacePolicy` | Data lost when an update replaces a resource |
| `Stack policy` | Updates to named resources unless overridden |
| `Termination protection` | Deleting the whole stack by mistake |

> ON THE EXAM
>
> Production databases: Snapshot or Retain on both policies, plus a stack policy.

# Secrets in infrastructure code

| APPROACH | VERDICT |
| --- | --- |
| Password in template or parameter file | Never: stored in source control |
| NoEcho parameter | Hidden in console, still supplied as plain text |
| Dynamic reference to Secrets Manager | Resolved at deploy time, never stored |
| SSM SecureString dynamic reference | For supported properties and simple values |
| RDS ManageMasterUserPassword | Database generates and rotates its own secret |

ON THE EXAM

No human should ever see the database password: let RDS manage it in Secrets Manager.

# StackSets permission models

SELF-MANAGED

## You create the roles

- Administration and execution roles per account
- List target accounts explicitly
- Works without Organizations
- New accounts added by hand

SERVICE-MANAGED

## Organizations does it

- Target OUs, not account lists
- Roles created automatically
- Automatic deployment to new accounts
- Delegated administrator supported

THE TIEBREAKER

Every account in an OU, including new ones, with no manual steps: service-managed with auto-deployment.

# Reading the scenario

| WHEN THE QUESTION SAYS | IT IS TESTING |
|---|---|
| Loops, types, reusable components | AWS CDK |
| Password must never be seen or stored | ManageMasterUserPassword, dynamic references |
| Every account in an OU, including new ones | Service-managed StackSets, auto-deployment |
| Console changes should be reverted | Drift-aware change sets |
| Existing resources into IaC, no recreation | IaC generator and resource import |
| Database must survive template mistakes | Deletion and replace policies, stack policy |

ON THE EXAM

Code the infrastructure, keep secrets out, protect the data, and scale by OU.

> Meridian Freight's platform team maintains 40 near-identical CloudFormation templates for regional depots, edited by copy and paste. They want loops, type checking, unit tests and a shared library of company components.
>
> Security reviewers still want to review the deployed CloudFormation.

## What should the solutions architect recommend?

A. Keep YAML templates and add more nested stacks

B. Use the AWS CDK with a shared construct library, reviewing the synthesised templates

C. Write Python scripts that call the AWS SDK directly

D. Create resources in the console and document them

◆ PAUSE AND DECIDE

## What should the solutions architect recommend?

A More nested stacks

*Helps reuse, but still no loops, types or unit tests.*

B CDK with constructs

Programming features, shared components, and CloudFormation output.

C SDK scripts

*No state, no change sets, no rollback.*

D Console

*Not infrastructure as code at all.*

THE SIGNAL

Programming features with reviewable CloudFormation: the CDK.

> Meridian Freight's CloudFormation template creates an Aurora cluster. Today the master password is passed as a NoEcho parameter from a file in the pipeline.
>
> Auditors require that no person or file ever holds the password, and that it rotates automatically.

## What should the solutions architect change?

A Encrypt the parameter file with KMS

B Store the password as a plain Parameter Store String

C Hard-code the password in the template and restrict repository access

D Set ManageMasterUserPassword so Aurora generates and rotates the password in Secrets Manager

◆ PAUSE AND DECIDE

## What should the solutions architect change?

A Encrypted file

*A file still holds the password, and it does not rotate.*

B Plain String parameter

*Stored unencrypted, and still set by a person.*

C Hard-coded

*The worst place for a secret.*

D Managed master password

Generated and rotated by the service; nobody sees it.

THE SIGNAL

No person or file holds the secret, and it rotates: let the database manage it.

> Every account in Meridian Freight's Workloads OU needs a standard set of IAM roles, CloudWatch alarms and an SNS topic, in four Regions.
>
> New accounts are added weekly and must receive the baseline automatically; accounts that leave the OU should have it removed.

**What should the solutions architect implement?**

A. A CloudFormation StackSet with service-managed permissions targeting the OU, with automatic deployment enabled, in four Regions

B. A pipeline that deploys a stack into each account in turn

C. A self-managed StackSet with a list of account IDs

D. A CDK app run manually by each account owner

◆ PAUSE AND DECIDE

## What should the solutions architect implement?

**A** Service-managed StackSet

Targets the OU; deploys to joiners and removes from leavers.

**B** Pipeline per account

*Needs updating every week as accounts change.*

**C** Self-managed list

*New accounts must be added by hand.*

**D** Run by each owner

*Inconsistent and easily skipped.*

THE SIGNAL

Every account in an OU, automatically, joiners and leavers: service-managed StackSets.

> An engineer changed a production security group in the console during an incident, and a load balancer listener was deleted by hand. Both resources belong to a CloudFormation stack.
>
> The team wants the next deployment to restore the template's configuration, with a preview of what will change first.

**What should the solutions architect use?**

A. Delete and recreate the stack

B. A standard change set with an unchanged template

C. A drift-aware change set, previewed and then executed

D. Fix both resources by hand in the console

◆ PAUSE AND DECIDE

## What should the solutions architect use?

A Recreate the stack

*Disruptive, and risks data in other resources.*

B Standard change set

*Compares templates only; sees no difference, changes nothing.*

C Drift-aware change set

Compares template with actual state; restores and recreates.

D Manual fix

*More console changes, no preview, no record.*

THE SIGNAL

Revert out-of-band changes through IaC, with a preview: drift-aware change sets.

> Meridian Freight's analytics account holds about 200 resources created by hand over several years, including S3 buckets, IAM roles and Glue jobs. They must come under CloudFormation management.
>
> No resource may be deleted or recreated in the process.

## What should the solutions architect use?

A Write new templates and deploy them alongside the old resources

B The CloudFormation IaC generator to produce templates, then import the resources into stacks

C Delete the resources and redeploy them from new templates

D Tag the resources as managed

◆ PAUSE AND DECIDE

## What should the solutions architect use?

A New templates alongside

*Creates duplicates; the originals stay unmanaged.*

B IaC generator and import

Templates from what exists, adopted without recreation.

C Delete and redeploy

*Explicitly ruled out, and loses data.*

D Tags

*Labels, not management.*

THE SIGNAL

Bring existing resources under IaC without touching them: IaC generator and resource import.

> A Meridian Freight developer renamed a property in a template, which caused CloudFormation to replace the production RDS database. Data was lost.
>
> The team wants to make sure stack changes can never silently destroy the production database or its data.

**What should the solutions architect configure?**

A Enable termination protection only

B Move the database out of CloudFormation entirely

C Add a CloudWatch alarm on database connections

D Snapshot as both DeletionPolicy and UpdateReplacePolicy, a stack policy denying replacement of the database, and change set review

◆ PAUSE AND DECIDE

## What should the solutions architect configure?

A Termination protection

*Stops stack deletion, not resource replacement in an update.*

B Remove from IaC

*Gives up the benefits of IaC.*

C Connection alarm

*Reports the loss after it happens.*

D Policies plus review

Blocks replacement, keeps a snapshot, and surfaces it in previews.

THE SIGNAL

Replacement during an update is blocked by a stack policy, and caught by UpdateReplacePolicy.

# What to carry out of this module

- **CDK** for programming features, synthesising reviewable CloudFormation.
- **No secrets in templates**: managed passwords and dynamic references.
- **Protect data**: Deletion and UpdateReplace policies, stack policies.
- **Drift-aware change sets** revert; IaC generator and import adopt.
- **Service-managed StackSets** follow the OU, joiners and leavers.

> ON THE EXAM
>
> Code it, keep secrets out, protect the data, preview every change, scale by OU.